

Replacing `std::unordered_map` with an open-addressing hash map

Learning notes for the order-book `IdMap`. The book needs one lookup on the hot path: given an **order id**, find where that order lives (`order_id -> pool index`), so `cancel` and `modify` can reach it in $O(1)$. We started with `std::unordered_map<uint64_t, uint32_t>` and replaced it with a hand-rolled open-addressing map. This explains why, and how.

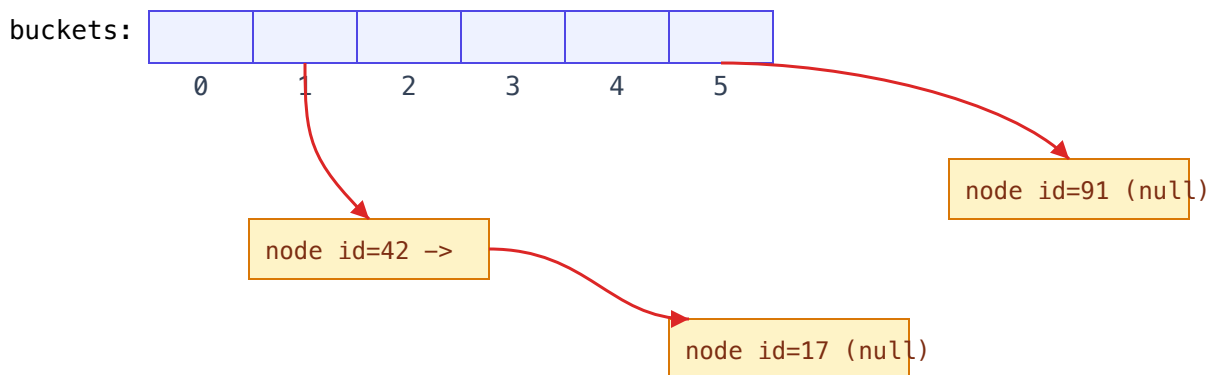
1. Why `std::unordered_map` is slow on a hot path

`std::unordered_map` is a **chained** hash table. The standard effectively forces:

- **A separate heap-allocated node per element.** Every `insert` calls the allocator; every `erase` frees. Allocation is slow and, worse, scatters nodes across memory.
- **Pointer chasing.** A bucket holds a pointer to a linked list of nodes. A lookup hashes the key, indexes the bucket array, then follows node pointers comparing keys. Each `next` is very likely a **cache miss** (the node is somewhere unrelated in memory).
- **Poor cache locality.** The keys you compare live in scattered nodes, not packed together, so the CPU's prefetcher can't help you.

On a trading hot path where you do millions of `cancel / modify` lookups per second, those cache misses dominate. A miss to main memory is $\sim 100+$ ns; an L1 hit is ~ 1 ns.

Chaining (`std::unordered_map`): bucket array + pointers to scattered heap nodes



red = pointer chase; each hop is likely a cache miss (~ 100 ns)

2. The idea: open addressing

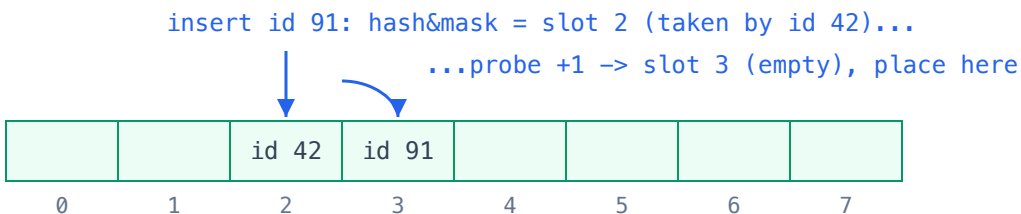
An **open-addressing** table stores entries **inline in one flat array** - no nodes, no pointers. When two keys hash to the same slot (a collision), you don't chain; you **probe** to another slot in the same array by a fixed rule. Our rule is **linear probing**: on a collision, try the next slot, then the next, wrapping around.

Because everything is in one contiguous array, a lookup touches only a couple of adjacent cache lines, and the prefetcher loves sequential scans. No allocator on the hot path.

```
enum State : uint8_t { EMPTY = 0, FULL = 1, DELETED = 2 };
struct Slot { uint64_t key; uint32_t val; uint8_t state; };
std::vector<Slot> slot_; // one flat array, power-of-2 size
```

The array size is a **power of two**, so `hash(key) & (cap - 1)` replaces a modulo (a slow integer division) with a single AND.

Open addressing + linear probing: everything inline in one flat array



find scans adjacent slots (cache-friendly); stop at the key (hit) or EMPTY (miss)

3. The hash function

Order ids are often sequential (1, 2, 3, ...) or come straight from an exchange. Feeding raw sequential integers into a table causes clustering. We run the key through the **splitmix64 finalizer** - three xor-shift-multiply steps that "avalanche" the bits so similar inputs land far apart, for a few cheap ALU ops (no memory access):

```
static uint64_t hash(uint64_t x) { // splitmix64 finalizer
    x ^= x >> 30; x *= 0xbf58476d1ce4e5b9ull;
    x ^= x >> 27; x *= 0x94d049bb133111ebull;
    x ^= x >> 31; return x;
}
```

4. Find (the hot path)

Probe forward from the home slot; stop at the key (hit) or an `EMPTY` slot (miss). An `EMPTY` slot means "nothing was ever placed at or past here in this probe run," so the key can't exist.

```
uint32_t find(uint64_t key) const {
    std::size_t i = hash(key) & (cap_ - 1);
    while (true) {
        const Slot& s = slot_[i];
        if (s.state == EMPTY) return 0xFFFFFFFFu;    // not present
        if (s.state == FULL && s.key == key) return s.val;
        i = (i + 1) & (cap_ - 1);                    // linear probe
    }
}
```

At load factor < 0.7 the average probe length is ~ 1 -2 slots, all cache-adjacent.

5. Deletion: why tombstones

You cannot just blank a slot on delete - that would insert a false `EMPTY` mid-probe-run and make later keys unfindable. Two standard fixes:

- **Tombstones (what we use):** mark the slot `DELETED`. `find` treats it as "keep probing"; `insert` may reuse it. Simple and correct. Downside: tombstones lengthen probe runs over time, so we **rehash** to clear them when the table gets crowded.
- **Backward-shift deletion:** on delete, shift subsequent displaced entries back to fill the gap, keeping the table tombstone-free. Faster steady-state, more code. A good "next step."

```
bool erase(uint64_t key) {
    std::size_t i = hash(key) & (cap_ - 1);
    while (true) {
        Slot& s = slot_[i];
        if (s.state == EMPTY) return false;
        if (s.state == FULL && s.key == key) { s.state = DELETED; --size_; ++tomb_; return true;
        i = (i + 1) & (cap_ - 1);
    }
}
```

6. Load factor and rehash

Probe length blows up as the table fills (that's the birthday-paradox tail). We keep `(live + tombstones) / capacity < 0.7`; when it crosses that, we allocate a 2x array and re-insert only the

live entries (which also purges tombstones). Rehash is $O(n)$ but rare, so amortized cost stays $O(1)$. `reserve(n)` presizes so a known workload never rehashes mid-run.

```
void insert(uint64_t key, uint32_t val) {
    if ((size_ + tomb_) * 10 >= cap_ * 7) rehash(cap_ << 1);    // load >= 0.7
    // ... linear-probe to the key (update) or first EMPTY/DELETED slot (place)
}
```

7. Why it's faster (the summary)

	<code>std::unordered_map</code>	open-addressing <code>IdMap</code>
memory	node per element, scattered	one flat array
allocation on hot path	yes (insert/erase)	none
lookup	hash + bucket + pointer chase	hash + short cache-adjacent scan
cache behavior	misses on node chains	prefetcher-friendly

Measured (this project, Apple clang -O3 -march=native): swapping the map lifted the book from ~14M to ~20.8M msg/s (speedup over the naive baseline 1.8x -> 2.7x), and on the real BTC replay it halved median op latency, 84 ns -> 42 ns.

8. Trade-offs (when NOT to bother)

- **Clustering:** linear probing forms runs; a bad hash makes them long. `splitmix64` mitigates.
- **Tombstone buildup** under delete-heavy churn - needs the rehash (or backward-shift).
- **Reserved key values:** open addressing needs sentinels; here `state` avoids stealing a key value, but a value-based sentinel scheme would reserve one key.
- For cold paths or small/rarely-touched maps, `std::unordered_map` is perfectly fine and saves you the code. This win only matters because it is on the microsecond hot path.

9. Where to go next

1. **Hand out pool indices as the id** where the feed lets you - then no lookup at all.
2. **Backward-shift deletion** to kill tombstones.

3. **Robin Hood hashing** or a **Swiss table** (SIMD probing, e.g. `absl::flat_hash_map`) for even tighter tails.
4. Measure with `perf` - confirm the lookup left the top of the flamegraph.